Budget Alert Feature

Overview

The Budget Alert feature allows users to **set monthly budgets for specific spending categories** and receive **alerts when their spending approaches or exceeds the budget**.

The system automatically checks spending whenever a receipt is uploaded and triggers alerts when:

- **80% of the budget is reached**
- **Budget is exceeded**

Alerts are delivered in two ways:

- **In-app notification**
- **Email notification to the registered user**

Feature Workflow

Step 1 – User sets a budget

Users can create a monthly budget for a category.

Example:

Category	Month	Budget
Groceries	June	₹100

Budgets are stored in the **MongoDB budgets collection**.

Example document:

```
{
  "user_id": "12345",
  "category": "groceries",
  "month": 6,
  "year": 2026,
  "monthly_budget": 100
}
```

Step 2 – Receipt upload

Users upload a receipt through the dashboard.

The backend performs:

1. **OCR extraction**
2. **AI parsing**
3. **Receipt storage**
4. **Budget validation**

Step 3 – Spending calculation

The system calculates **total spending for a category in a given month**.

File:

models/receipt.py

Function used:

```
def total_spent_by_category(self, user_id: str, category: str, month: int, year: int) -> float:
    start = f"{year:04d}-{month:02d}-01"

    if month == 12:
        end = f"{year + 1:04d}-01-01"
    else:
        end = f"{year:04d}-{month + 1:02d}-01"

    pipeline = [
        {
            "$match": {
            "user_id": user_id,
            "category": category,
            "receipt_date": {"$gte": start, "$lt": end}
        }
    },
    {
        "$group": {
            "_id": None,
            "total": {"$sum": "$total"}
        }
```

```
        }
    ]

    result = list(self.collection.aggregate(pipeline))

    return result[0]["total"] if result else 0.0
```

This function calculates **total spending for a category within a month**.

Step 4 – Budget validation

After a receipt is saved, the backend checks the user's budget.

File:

routes/receipts.py

Logic implemented:

```
budget = budgets_collection.find_one({
    "user_id": user_id,
    "category": category,
    "month": month,
    "year": year
})
```

If a budget exists, spending is compared.

Step 5 – Alert conditions

Two alert levels were implemented.

80% Budget Warning

Triggered when:

spent >= 0.8 * budget

Example message:

⚠ Groceries spending reached 80% of budget.

Budget Exceeded Alert

Triggered when:

spent > budget

Example message:

⚠ Budget exceeded for groceries in 6/2026.

Spent ₹272 of ₹100

Step 6 – Email notification

When a warning or exceeded condition occurs, the system sends an email to the user.

File created:

utils/email_service.py

(pushed)

Step 7 – Mail configuration

Configured in:

app.py

```
from flask_mail import Mail

app.config["MAIL_SERVER"] = "smtp.gmail.com"
app.config["MAIL_PORT"] = 587
app.config["MAIL_USE_TLS"] = True
app.config["MAIL_USERNAME"] = "algosphere.alerts@gmail.com"
app.config["MAIL_PASSWORD"] = "APP_PASSWORD"
app.config["MAIL_DEFAULT_SENDER"] = "algosphere.alerts@gmail.com"

mail = Mail(app)
```

Email alerts are sent through Gmail SMTP.

Step 8 – User email retrieval

User email is retrieved from MongoDB.

```
from bson import ObjectId

user = db.users.find_one({"_id": ObjectId(user_id)})
user_email = user["email"] if user else None
```

The email is passed to the alert service.

Step 9 – API response

The receipt upload API returns alert information.

Example response:

```
{
  "receipt": {...},
  "alert": "⚠ Budget exceeded for groceries",
  "spent": 272
}
```

This allows the frontend to display alerts in the dashboard.

Files to Push to GitHub

You should push the following files.

Backend

app.py

routes/receipts.py

routes/budgets.py

models/receipt.py

models/budget.py

utils/email_service.py

Frontend (if modified)

dashboard.html

dashboard.js

budget_manager.js

styles.css

Feature Summary

The Budget Alert system now supports:

- Category-based budgeting

- Monthly budget tracking
- Automatic spending calculation
- 80% warning alerts
- Budget exceeded alerts
- Email notifications
- Dashboard spending overview

This feature enhances the application by enabling **proactive expense management and automated financial alerts**.